DAMAGE-STAGES — our damage formula against the authority, stage by stage

Version: 5.205.0 — 2026-08-28.

5.205.0 — THE SPRINT IS PAUSED AND THIS FILE IS RE-READ, NOT REWRITTEN: NO STAGE MOVED. The living-docs deferral that ran from 2026-08-10 is over. Across the whole sprint the stage ORDER inside one hit is unchanged, and `tests/test-damage-stages.js` re-reads **1696/1696 exact, 0 at the wrong stage**, across all sixteen rolls and both crit states, with 5 re-derived `CH_EXACT` overrides and 0 wrong. The population moved (this file previously recorded 1728 cells); the verdict did not.

WHAT CHANGED IS OUTSIDE THIS CHAIN, AND BOTH ITEMS BELONG TO THE BATTLE LOOP. The crit is now drawn **once per hit** rather than once per click, which is what the authority does — its loop is the Champions mod's and its die is mainline's, inside the per-hit `getDamage` call, and neither `getSpreadDamage` nor `getDamage` is overridden by the mod. Arrival 2 of a volley no longer inherits arrival 1's crit. Separately, the event die itself gained a finalising mix; before that, consecutive arrivals shared a 16-bucket damage index 89.5% of the time against a correct 6.25%. **Neither touches the stage order this document describes**, and both are why a damage figure measured before 2026-08-27 is void rather than stale.

THE DIFFERENTIAL THAT CHECKS THIS CHAIN HAS NEVER APPLIED A MULTI-HIT MOVE, AND THAT MUST BE SAID WHEREVER ITS FIGURE APPEARS. Read from `data/engine-diff.json`: 6000 compared, 6000 agreed, 0 disagreed, and 0 at each of the sixteen band indices separately. Its own `scope` field limits it to damage only — no items, no abilities, no turn order, no status duration, no switching — and it records `skipped_multihit` 134 and `skipped_ability_multihit` 17, because the harness calls the authority's single-hit entry point rather than the volley loop. The multi-hit defects corrected during this sprint were invisible to it by construction. **The interior of a multi-hit range remains a single draw across the summed endpoints** rather than N independent ones; that is unchanged by this pass and is still the battle loop's question rather than this chain's.

3.98.0 — NO STAGE MOVED AND NOTHING IN THIS FILE CHANGED. ROADMAP #126 wired Quick Guard onto the priority-refusal gate. That gate sits in the TURN LOOP, above the action-kind dispatch, and never reaches `dmgRange`: a refused move deals no damage at all rather than damage at a different stage. `tests/test-damage-stages.js` is unchanged. The version moves because the CHANGELOG top moved, and this line says why the content did not.

3.97.0 — THE DAMAGE IS A LOOP OVER HITS NOW, AND NO STAGE MOVED. `dmgRange` is a wrapper over `dmgRangeOneHit`; the stage ORDER inside one hit is byte-for-byte what this document already describes, and `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage**. What changed is how many times that chain is spent: once per HIT for a move whose base power is a function of the hit index (Triple Axel `20 * move.hit`, Beat Up one packet per eligible ally), and once in total — with the old `_hits` scalar — for everything else, including the rest of the multi-hit family. **The pinned endpoints are the authority's**: `min` is every hit at the 85% randomizer and `max` every hit at 100%, which is what a pin produces in Showdown. **The interior of a multi-hit range is still a single draw** across the summed endpoints rather than N independent ones — unchanged by this pass, and it is the battle loop's question rather than this chain's. Fickle Beam's conditional power left this file's arithmetic entirely: it is a DRAW taken in the battle loop, not a ×1.3 on the base power.

3.96.0 — THREE DEAD LINES LEFT THE ATTACK AND DEFENCE CHAINS, AND ONE LIVE ONE JOINED. The hardcoded Choice Band / Choice Specs / Assault Vest multipliers were permanently false — all three are banned in this format — and are replaced by a derived `statMult` consumer whose only live member here is Light Ball. The chains themselves are unchanged in ORDER and `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage. The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.95.0 — A DAMAGE ANSWER CHANGED, AND IT IS NOT A STAGE. `dmgRange` now returns **0** against an intact Disguise, because the authority's `onDamage` blocks the move outright and the `maxhp/8` that busts it belongs to the ABILITY. Nothing in the table below moved — no multiplier changed position, and `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage** — but this is the first entry here that alters what the function RETURNS rather than where a multiplier applies, so it is recorded as such. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.94.0 — NOTHING IN THE TABLE BELOW MOVED. ROADMAP #110's `selfBoost` fix adds the USER's own stat change to two move rows; it is a boost applied after the move, not a multiplier inside it. `tests/test-damage-stages.js` re-reads **1728/1728 exact, 0 at the wrong stage. The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.93.0 — NOTHING IN THE TABLE BELOW MOVED. ROADMAP #110's partial-trap fix is a duration COUNTER, not a multiplier, and the chip fraction it carries (1/8) was correct throughout — what changed is that it is now derived from `onStart`'s `boundDivisor` rather than typed. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.92.0 — NOTHING IN THE TABLE BELOW MOVED. `tests/test-damage-stages.js` stopped padding its inert slots with `Tackle`, which is `isNonstandard: 'Past'` and not in this format. Those slots never act, so the reading is unchanged: **1728/1728 exact, 0 at the wrong stage. The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.91.0 — NOTHING IN THE TABLE BELOW MOVED, AND NO STAGE QUESTION WAS RAISED. ROADMAP #116 is a legality guard on the probe harness, not a multiplier. It changes which bodies may be staged, never where a multiplier applies. Worth one line here for a reason that touches this document directly: the stage table is measured by probes, so a probe staging an entity the format does not contain would put a row in it about a mechanic no game can reach. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.90.0 — NOTHING IN THE TABLE BELOW MOVED, AND THE ONE STAGE QUESTION THIS PASS RAISED WAS ANSWERED NO. ROADMAP #103 is a hit COUNT, not a stage. The plausible alternative WAS a stage question — whether the authority's per-hit floor (`n` independent `floor`s) differs from this engine's single `floor(v * n)` — and it was ruled out with arithmetic rather than a preference: `roll()` already returns an integer, so for an integer count the two expressions are equal for every value. The multiplication at `dmgRange`'s tail is unchanged. **The boost-stage difference filed at 3.89.0 is still filed and still not fixed.**

3.89.0 — NOTHING IN THE TABLE BELOW MOVED, AND ONE NEW DIFFERENCE IS FILED INTO IT. ROADMAP #101 and #102 are a post-damage REACTION and a HEAL; neither is a stage. But wiring Strength Sap's heal off `getStat('atk', false, true)` surfaced a boost-stage difference that IS a damage question: `dmgRange` applies a stage as `Math.floor(x * boostMul(s))`, where `sim/pokemon.ts` MULTIPLIES on a positive stage and **DIVIDES** on a negative one. The two disagree wherever the float lands just under an integer — at `s = -1, x = 3` they give 1 and 2. The new heal uses a helper (`statWithBoost`) that mirrors the authority exactly; `dmgRange` was deliberately left alone in the same pass, because changing it is a damage change and would have needed its own measurement. **Filed here, not fixed.**

3.88.0 — TWELVE MOVES WERE PRICED OFF GENERIC GEN-9 DATA INSTEAD OF THIS FORMAT'S, AND THE BUILDER THAT FIXED THEM WAS ONE RUN AWAY FROM DELETING TEN SPECIES. Trop Kick read 70 where the format says 85, Mountain Gale 100 against 120 — ours low in all twelve, and MAG's own table had the right numbers the whole time, so the two engines disagreed on every one. Asking what a regeneration WOULD do, before running one, turned up 788 destructive changes waiting in the same builder and a header stamp whose regex had never once matched. `buffsHolderOnHit` also gained its condition by derivation — Anger Point only on a critical hit, Justified only on Dark — but **the engine does not read it yet and nothing behaves differently**, which is said here rather than left to look like a fix.

3.87.0 — THE STAGE WAS RIGHT AND THE INPUT TO IT WAS NOT: TWO READERS OF THE WEATHER. Nothing in the table below moved. The BasePower-chain member at `:1991` (`weatherScaled`) sits at the correct stage and reads the correct sky, because `dmgRange` shadows the field through `effWeatherOf` at its first statement — which applies the PRIVATE weather a Mega Sol body carries. The battle loop's own type authority, `effMoveType` , did not: it read `field.weather` raw. So the same click was a Fire move to this table and a Normal one to the stage-5 immunity gate, and a Meganium-Mega's Weather Ball priced here at 128-151 dealt 0 to a Ghost. `effMoveType` now CALLS `effWeatherOf` . This is §2's lesson in a new place: the stage-by-stage audit can be clean while a DIFFERENT reader of the same fact makes the whole row unreachable.

3.86.0 — EVERY PUBLISHED ARTIFACT HAS A WRITER NOW, INCLUDING THE ARM MILTANK ACTUALLY RUNS. The tool that answers "is this number still true" could only find a writer by an artifact's literal name beside a write call in two directories — so the mechanics census, the game differential, the interaction matrix and the deliberate roster, which are the four clauses of the MEDICHAM gate, had no row at all. Not ok, not unsafe: absent. The graph goes 115 to 160 artifacts and the unknown set 61 to 16, membership derived through four ranked arms that each record how they matched. UNSAFE rises 13 to 20 because seven artifacts that were always unsafe are now visible; none left the set.

3.85.0 — THE WHOLE SITE WITHHOLDS NOW, AND THE DEPLOYED COPY WAS MISSING THE FILE THAT MAKES IT WORK. Five pages LOADED a quarantined artifact as data instead of quoting its verdict, so the citation checker could not see them at all; seven of the Stadium's fifteen cabinets now go dark, each keeping its seat and its button and answering with the quarantine instead of a number. The file that drives all of it, `quarantine-data.js` , did not exist under `app/` — which is the copy a visitor loads — so every guard there took the healthy path. That is the same failure as the day before, one directory over.

3.83.0 — THE PINCH FAMILY FIRES, AND THE REFUSAL THAT HID IT WAS CORRECT THE WHOLE TIME. 3.80.0 (below) found that Blaze, Torrent, Overgrow and Swarm carry their below-1/3-HP condition as PROSE and that the `damageBoost` consumer refuses any condition it cannot evaluate. That refusal is #92's own rule and it is not removed. What changed is upstream: `engine/tag_dex.js` now derives the gate by SHAPE out of Showdown's `attacker.hp <= attacker.maxhp / 3` into `{cond:'hpFraction', of:'self', cmp:'<=', num:1, den:3}` , and `condHolds` evaluates it in INTEGER arithmetic — `maxhp * (1/3)` is not `maxhp / 3` , and a body at exactly one third would be refused a boost it is owed. An `onlyWhen` the engine still cannot read returns null, still refuses, and is now counted. The narrowed shape's membership went 5 → 9 and `tests/test-damage-stages.js` still reads 1728/1728 exact with 0 at the wrong stage. 9,141 uses, on today's corpus.

ROADMAP #88 AND #91 — ONE PIN WAS ONE CORNER, AND A CLICK WAS COUNTED AS A TEST (3.73.0). Every die in the differential was pinned a single way, which bought determinism — any difference is a bug, no statistics — and paid for it in coverage nobody had priced. The speed tie always resolved the same direction, every move below 100 accuracy MISSED ON BOTH SIDES, and damage was always the maximum roll, which is the one roll where the crit's wrong position happened to come out right. Rock Slide had never connected in this instrument; under the new arms it misses in one and hits in another, and a crit lands in the bottom arm and not the top. The pin set is now a declared run parameter, digested into `mode` , and a before/after pair whose pins differ is REFUSED rather than reported. Separately, coverage credit moved from the CLICK to the OBSERVED EFFECT: the old rule incremented when an entity was clicked and never asked whether the move did anything, so Haze clicked into a board with no boosts on it — a no-op — marked Haze exercised and stopped the steering selecting it. Five rows were clicked-or-present and did nothing at all: `critDamageUp` , `preventsSwitch` , `privateWeather` , `clearsScreens` and `preTurnShield` . The old rule called all five covered. **THE BASELINE IS RESET: both changes alter which games get played, so no run after this is comparable with the turn-1 figure published at 3.71.0 or with** `data/state-ladder.json` **.** And an ENGINE defect fell out of the tie work, filed rather than fixed here: the two engines have disagreed about EVERY speed tie for the life of this instrument — the authority resolves a tie to the LATER body in input order, `sortTurnOrder` draws one tie value per action from a constant scalar so the sort is stable and takes the EARLIER one. The instrument's own header claimed the pin made them agree by construction; that claim was false and was repeated as fact before it was checked. `sortTurnOrder` is the live engine, not instrument code.

3.82.0 — THE FIRST ENGINE FIX OF THE QUEUE LANDS: THE VOLATILE DURATION FAMILY, 9,092 USES, AND IT WAS THE PERISH SONG BUG A SECOND TIME. Showdown decrements a volatile's duration inside the Residual event, so one applied on turn N has already spent a turn by the end of it. That defect was documented in this engine for Perish Song, fixed for Perish Song, and left standing for every other duration-bearing volatile. Taunt and Disable now match the official engine; Encore's counter row is gone and only a separate HP row remains. Whole-game board agreement rose 76.9% to 78.9% on a paired differential, the roster's moves queue fell from 52 to 50, and the census did not move. The previously published baseline could not be reproduced because the census digest and the team store had both shifted underneath it — so the run was discarded and re-taken paired. The delta is the measurement.

3.81.0 — THE QUARANTINE REACHED THE BOARD, AND THE CHECKER THAT POLICES IT WAS BLIND TO THE PUBLISHED COPY. Thirteen slots on ABRA WORLD's status board now render as a redaction bar rather than a number, each carrying the artifact, the reason and the command that re-runs it. Two defects in the guard itself were found doing it: its own selftest had gone red — an `all` -stage artifact was matching ANY requested stage name, so "a missing stage must FAIL" stopped being enforced — and its citation walker looked at docs/ and web/ but never at app/, which is the copy a visitor actually loads. Five withheld verdicts were being published from app/ the whole time the check read green.

3.80.0 — THE DELIBERATE ROSTER'S INERT BUCKET COLLAPSED BY 94.1% OF ITS USAGE, AND A FAMILY OF ABILITIES WORTH 8,524 USES TURNS OUT NEVER TO HAVE FIRED. 124 abilities were falling through a catch-all that stages a plain attack, so the condition each one needs was never created and the roster honestly reported INERT — which reads as "nothing to test" when the truth is "never tested". Fifteen new shape rules take that bucket to 59 abilities / 4,261 uses, all 22 ability rules caught their own break, and nothing left in the bucket is above 500 uses. The first real defect it found: Blaze, Torrent, Overgrow and Swarm carry their below-1/3-HP condition as PROSE, and the consumer refuses any condition it cannot evaluate — so the pinch family has never once fired. The roster's artifacts are now written per stage, so the quarantine gate reads measurement rather than absence.

3.79.0 — EVERY FIGURE DOWNSTREAM OF MEDICHAM IS NOW WITHHELD RATHER THAN CAPTIONED, AND THE DELIBERATE ROSTER'S MOVES STAGE RAN FOR THE FIRST TIME. Will's standing call: *"all engines that take medicham's output should be regarded as out of date and we should stop referencing them until medicham is up to date and we can rerun them."* 34 of 114 artifacts are downstream and are no longer printed at all — R1, R2, R3, R4, leaf calibration and the weights among them — because a caption is not a quarantine: `PRE-CHANGE` had been printed beside those numbers for days and they went on being quoted anyway. Membership is derived from the dependency graph, not typed, and the gate that lifts it is computed

from the differential and the roster, where a MISSING stage counts as failing. The roster gained 26 move shape rules and staged all 500 legal moves for the first time, returning a 79-row queue over ~15,000 uses. Its control arm was found to be measuring the CONTROL rather than the subject, which made six ability findings false; **Weather Ball, Sand Rush and Damp are retracted as defects and are correct.**

3.78.0 — THE SHEET'S REAL NATURE NOW REACHES BOTH ENGINES, AND THE TURN-1 NUMBER FELL. THAT IS THE INSTRUMENT GETTING HONEST. The whole-game differential built every body `Serious` while the stored sheet beside it said `Modest`, and with every body flat AND Serious, 326 of 357 species in the format share a Speed with some other species — so the rig MANUFACTURED speed ties and almost never tested a real speed differential. Carrying the declared nature cut the tied groups the resolver has to break from 348,595 to 243,467 over the same 1,998 games, a 30.2% fall. The instrument's own numbers went DOWN, as predicted before the run: the board at the end of turn 1 is identical in 97.4% of games flat against 97.3% natured, games whose board never parted 80.8% against 78.8%, and the median turn of first board divergence one turn earlier at 7. Encore divergences nearly doubled (10 to 19 games), which is what a duration volatile that only bites when turn order does looks like when turn order starts being tested. THE SPREADS REMAIN ABSENT AND ALWAYS WILL BE — a Showdown open team sheet does not reveal them (`"evs": null` on 173,784 of 173,784 stored bodies), so this narrows the declared gap and does not close it. Neither engine is told the other's answer: both are told the nature and each computes, and the alignment assertion still reads 0. It read 21 on the first run and all 21 were Ditto — entry-time Imposter had already transformed the medicham body, and the harness was writing the copied stat line onto Showdown's Ditto before the game began. Census 324 live, 0 missing, unchanged.

3.77.0 — CONFUSION DID NOT EXIST, AND BURN HAD NEVER BEEN ON A BOARD. The confusion volatile was written and never read or ticked, so Hurricane's secondary — 3,779 uses — fell through every branch, and the two berries that clear confusion looked dead because there was nothing to clear. The sleep counter was an ordering bug: the authority runs sleep before flinch, ours ran flinch first, so a body that was asleep AND flinched never ticked and woke a full turn late. Burn, by contrast, is CORRECT and was confirmed rather than changed — but it had never once been staged, because Will-O-Wisp is 85-accurate and the harness pin makes every sub-100 move miss. The freeze timer is correct too; what was missing was the instrument, which carried no freeze counter at all, so the engine's value could drift and no measurement would see it. Census 324 live, 0 missing.

3.77.0 — THE ACROSS-A-SWITCH ARM FOUND A DEFECT ONE DAY OLD THAT FIXING SOMETHING ELSE CREATED. A transform never reverts when the body leaves the field: the authority clears it in `clearVolatile`, and this engine sets the flag and never unsets it. Since the transform also overwrites the body's name, stats, types, moves, boosts and ability, a benched Ditto is PERMANENTLY the thing it copied — so the two engines then choose replacements from benches that no longer describe the same Pokemon, and worse, a Ditto can only ever transform ONCE PER BATTLE, because the guard refuses a second. Re-copying is the entire function of the Pokemon. Imposter first fired the day before, and the out-and-back scenario that exposes this only became expressible hours earlier. The roster's two owed arms — across a switch, and at the exact HP line — are both built, both red-demonstrated, and Speed Boost and Focus Sash both match.

3.77.0 — A STAGED SCENARIO CAN NOW SWITCH, SO A MID-TURN ENTRANT IS EXPRESSIBLE FOR THE FIRST TIME. The scenario driver understood only a move; every other step became a pass, so no staged test could put a body on the field part-way through a turn. That single gap blocked three things at once: Speed Boost's entry gate, which exists only for a body that just switched in; Hunger Switch's flip and Zero to Hero's switch-out transform; and the whole across-a-switch arm of the roster. Four of the six engine defects found the day before were about a MOMENT rather than an effect, and no scenario without an entrant can express one. Verified end to end: Espathra switches in and reads +0 Speed in both engines on the turn it arrives, then +1 at the end of the next, with all 131 fields identical on both boundaries.

3.77.0 — ALL 316 ABILITIES STAGED DELIBERATELY, AND A FREE +6 ATTACK FELL OUT. Anger Point and Justified are one defect twice: a conditional boost-on-being-hit whose condition is never checked, so Anger Point grants +6 Attack off an ordinary hit where it requires a crit, and Justified grants +1 off a Poison move where it requires Dark. Hustle applies no 1.5x Attack at all. Two facts about the instrument matter as much: Gastro Acid does not suppress an ability here, and since suppression is the ONLY control available to 23 abilities, checking that control against a known-live fixture is what stopped five more from being published as dead for the control's failure rather than their own. And a fact about the regulation rather than the simulator: 113 of 316 legal abilities have NO legal carrier, so the effective roster of this format is about 203.

3.77.0 — FIVE MECHANICS THAT DID NOTHING, AND ONE THAT WAS ALREADY RIGHT. Imposter never transformed Ditto; Hunger Switch never flipped Morpeko; Knock Off took its 1.5x against an item it cannot remove; Fling never became an attack at all, because a base power of 0 made the click fail a `hasPower()` gate; and Roar's phaze branch held a Pokemon-first target, so a phaze after a pivot dragged nobody — the SIXTH site missed by the slot-first sweep, and at priority -6 the worst possible place to hold a body rather than a slot. Mawile's mega ability swap, which had been blamed for a whole family of Attack-stage divergences, WAS ALREADY CORRECT: the scenario was board-identical on its first run, and deleting the swap deliberately parts two fields at once, so the symptoms were real symptoms of a bug this engine does not have. Census 319/319 live, 0 missing; the staged harness now carries 24 scenarios, all clean and all breakable.

3.77.0 — THE INSTRUMENT RESOLVED A SWITCH BY TWO DIFFERENT KEYS AND FAILED SILENTLY BOTH WAYS. The driver names a bench member by Showdown's species id; the Showdown side looked it up by species id and the medicham side by the body's DISPLAY NAME. Those agree until a body is renamed — which this engine began doing the day before, when Disguise started renaming a busted Mimikyu, Zero to Hero started renaming Palafin, and Hunger Switch was queued to flip Morpeko every turn. After a rename the two keys part and that body can never be switched to again. Neither side raised anything: an unresolved lookup answered `pass` on both, so one engine could switch while the other stood still, producing a different board with no evidence attached. The key is now stamped at build time from the same expression the driver uses, and a miss is counted and printed beside the other declared gaps (0/0 over 120 games). This is an INSTRUMENT change rather than an engine one, so it alters what a measurement sees; it was also LATENT UNTIL THE FORME FIXES LANDED, and the deliberate-roster build would have walked into it.

> ***NO STAGE MOVED IN 3.77.0, AND THE VERSION MOVED ANYWAY — the reason is worth stating rather than pinning.*** *WIRE 139 changed WHICH BODY a move resolves against (the slot, not the Pokemon, which is what `Battle#getTarget` does), and WIRE 140 added Ally Switch, which moves two bodies between slots mid-turn. Neither touches a multiplier or its stage, so every row in the table below still holds exactly as measured — but both sit UPSTREAM of the whole table: a multiplier applied at the right stage to the wrong defender is wrong for a reason this document cannot see. Said here so a later reader does not conclude the audit was re-run.*

> ***WIRE 133–138 ADDED ONE MULTIPLIER TO THIS AUDIT AND SETTLED THE SPEED-TIE PARAGRAPH ABOVE (3.74.0).*** *The paragraph at the head of this file was RIGHT that the two engines disagreed about every speed tie and RIGHT that `sortTurnOrder` is the live engine, and its DIAGNOSIS was incomplete: "the authority resolves a tie to the LATER body in input order" is what the authority PRODUCES UNDER THIS HARNESS'S PIN, which replaces `PRNG.shuffle` with a no-op — it is not a rule. The rule is `Battle#speedSort`, a SELECTION SORT whose swaps move UNTIED elements around, ending in a Fisher-Yates over the tied group: a speed tie is a COIN FLIP. The engine now performs the same selection sort and resolves the residual group with the per-action uniform key it already drew, so both engines land on the same body under identical pinned dice and both are a fair coin under real ones.* `tests/test-speed-tie.js` *proves it in both team orientations, with the tied pair on one side, on a three-way tie, and against*

> *a no-tie control.* ***The one change to the DAMAGE formula itself is WONDER ROOM*** *(* `swapsDefences` *, 11 uses), which had no consumer at all. It swaps the STORED DEFENSIVE STAT and NOT the boost stage —* `Pokemon#getStat` *swaps* `storedStats` *at the top of the function and then applies* `boosts[statName]` *, the ORIGINAL stat's stage — so the swap is applied to* `D` *before the stage multiplier and* `_dKey` *is deliberately left unrewritten. It rides on whichever defence the move attacks into, so Psyshock and Body Press inherit it through* `statSwap` *rather than through a second rule. Nothing else in §2a or §3 moved.*

> ***THIS AUDIT HAS BEEN LANDED. ROADMAP #92, 3.73.0.*** *Everything §6 lists as open work is fixed, and the numbers below are now HISTORY — they describe the engine at release* `dc3c43336539` *, not the engine in the tree.* ***Read §2a and §3 as the record of what was wrong, not as a description of what is wrong.*** *What replaced them: - every* `onBasePower` *member is folded into ONE relay spent once, and every* `onModifyAtk` */* `onModifySpA` */* `onModifyDef` */* `onModifySpD` *member into two more — the STAGE and the CHAIN halves of §2's finding, which had to move together; - Friend Guard is inside the ModifyDamage chain rather than beside it; Helping Hand and the ally multiplier reach* `dmgRange` *on a seventh argument, because what it cannot DERIVE it can be TOLD; - the rolled crit's x1.5 is applied inside* `dmgRange` *before the randomizer, where the authority applies it, and Sniper has left that multiply for the final chain; - the four field terrains exist for the first time, with the authority's own grounded subject.* ***The claim is checked rather than asserted:*** `tests/test-damage-stages.js` *runs 54 scenarios × 16 damage rolls × 2 crit states against Showdown's own* `moveHit` *and demands EXACT equality —* ***1,728/1,728*** *— and it was shown RED on two deliberate reversions before being trusted. The census carries five of these as probes (* `move|setsTerrain` *×4,* `ability|damageBoost` *), which is what the last paragraph of this header said it did not yet do.* ***§2c is the part that keeps its original force****: it is the list of things that were checked and found CORRECT, and the gate now re-checks every one of them on every run. Four things are still not fixed and each is named with its reason in* `docs/ENGINE.md` *— Charge (no volatile exists to read),* `terrainScaled` *'s grounded SUBJECT (the tag carries none), Rivalry (no gender in* `MC.mons` *), and the artifact storing 1.3 as a float where the authority spells* `[5325,4096]` *(the engine carries a four-entry override that the gate re-derives from the live dex).*

Audited against engine release `dc3c43336539` and the Showdown checkout at `SHOWDOWN_PATH`. Every rate in this document was measured in the session that wrote it.

Will, 2026-08-07: *"LETS CHECK THE DAMAGE FORMULA FOR ALL ITS COMPONENTS AND COMPARE OURS AGAINST SHOWDOWN."*

Read against the frozen release `dc3c43336539` , not the live tree — `engine/medicham2-browser.js` was being edited by another agent while this was written. Every line number in the "ours" column is a line in `data/releases/dc3c43336539/engine/medicham2-browser.js` . Every line number in the authority column is `pokemon-showdown/sim/battle-actions.ts` or `sim/battle.ts` .

This document is an AUDIT. It changes no engine file and lands no mechanic. Nothing here is a probe in `tests/test-mechanics.js` yet, so nothing here is carried by the census — that is the next pass's job.

---

0. THE ANSWER THAT WAS ASKED FOR FIRST — FAIRY AURA IS A BASE POWER MULTIPLIER

`pokemon-showdown/data/abilities.ts` — `fairyaura.onAnyBasePower` , priority **20**:

```
if (target === source || move.category === "Status" || move.type !== "Fairy") return;
if (!move.auraBooster?.hasAbility("Fairy Aura")) move.auraBooster = this.effectState.target;
if (move.auraBooster !== this.effectState.target) return;
return this.chainModify([move.hasAuraBreak ? 3072 : 5448, 4096]);
```

Three facts, all read out of the handler rather than remembered:

1. **The stage is** `BasePower` **, not** `ModifyDamage` **.** Dark Aura is the same handler with `"Dark"` .
1. **The multiplier is** `[5448, 4096]` **, not** `1.33` **.** Our tag artifact carries `auraBoost.mult = 1.33` , and `trunc(1.33 * 4096) = 5447` — one 4096th low. Pass the pair to `md4096` / `ch4096` , which already accepts `[num, den]` for exactly this reason (its own header records the same trap for Tough Claws, `1.3 -> 5324` against the authority's `[5325, 4096]`).
1. **Aura Break does not suppress the aura — it INVERTS it** to `[3072, 4096]` (x0.75) at the same stage. Measured: Sylveon Moonblast into Goodra, no aura 98, Fairy Aura 132, Fairy Aura + defender Aura Break **74**. `aurabreak` has no entry in the tag artifact at all.

The cost of landing it at the wrong stage, measured

Eight Fairy-move rows, bodies flat L50/0EV/31IV/Serious in both engines, top roll. Two candidate fixes scored against Showdown: the multiplier applied to BASE POWER, and the same multiplier spent on the FINAL damage.

row	Showdown	ours today (no aura)	fix at BasePower	fix at ModifyDamage
Sylveon Moonblast -> Goodra	132	98	**132**	130
Sylveon Dazzling Gleam -> Snorlax	72	55	**72**	73
Gardevoir Moonblast -> Snorlax	94	72	**94**	96
Clefable Play Rough -> Goodra	162	122	**162**	162
Azumarill Play Rough -> Snorlax	67	51	**67**	68
Sylveon Draining Kiss -> Goodra	72	54	**72**	72
Gardevoir Dazzling Gleam -> Goodra	122	96	**122**	128
Clefable Moonblast -> Heliolisk	85	66	**85**	88

BasePower-stage fix: 8/8 correct. ModifyDamage-stage fix: 2/8. The two candidate fixes agree with each other on 2/8, so a wrong-stage aura is not "close enough" — it is wrong three quarters of the time, by up to 6 points on a 122-point hit.

The usage argument, and `uses` **is a sheet count so it is read carefully.** The tag artifact says `fairyaura: uses 0` . That figure is worthless here for the reason `docs/LESSONS.md` 3 gives: the ability is Gardevoir-Mega's, so it never appears in a sheet's ability slot. The real exposure is the STONE, times **every Fairy move either side clicks while it is on the field** — `appliesToEveryone: true` in our own artifact.

Exposure is read live from `data/tags.json` and is deliberately NOT restated here. The counts that sat on this line were true when it was written and were stale within hours — `tags.json` is regenerated whenever the tagger runs, and every one of them had moved by the next gate run. What matters and does not drift: the stone is on the order of hundreds of sheets, Moonblast is the most-clicked Fairy move in the corpus by a wide margin with Dazzling Gleam second, and the aura applies to EVERY Fairy move on the field rather than only the holder's — so the exposure is the stone's sheets multiplied by every Fairy click by anyone, which is why `fairyaura: uses 0` was worthless evidence. Read the current numbers out of the artifact.

---

1. THE AUTHORITY'S PIPELINE, ONE ROW PER STAGE

`getDamage` (`sim/battle-actions.ts:1585`) then `modifyDamage` (`:1724`). Three ways the authority applies a multiplier, and the difference between them is where every finding in this document lives:

how	what it is
`modify(v, m)`	`sim/battle.ts:2329` — `tr((tr(v * tr(m*4096)) + 2047) / 4096)` . Fixed point, round-half-up on 4096ths.
`chainModify` / `runEvent`	`battle.ts:2318` / `2302` — each handler folds into `event.modifier` ; the chain is spent ONCE by `finalModify` -> `modify` . **Two handlers in one chain truncate once, not twice.**
plain `tr()`	a bare truncated multiply. The source labels two of these "not a modifier" **on purpose**.

#	stage	authority (line)	how	ours (frozen line)	verdict
1	**BasePower chain** — items, abilities, terrain, Helping Hand, Charge, auras, the move's own `onBasePower`	`battle-actions.ts:1650` `runEvent('BasePower')` , then `clampIntRange(bp,1)` at `:1653`	chainModify, spent once	scattered: `:1985` (-ate), `:1991` (weatherScaled), `:1997` , `:2008-2112` (variablePower), `:2121-2137` (conditionalPower)	**PARTLY SAME STAGE** — the move-side members are here and correct; every ITEM and ABILITY member is not (see §2)
2	**Tera 60-BP floor**	`:1660-1667`	assignment	absent	ABSENT — no Tera in this engine at all; out of scope, stated
3	**stat modifiers** `ModifyAtk/SpA/Def/SpD`	`:1708-1709`	chainModify, spent once	`:2192-2245` through `md4096`	**SAME STAGE** for Choice items, Guts, Solar Power, Orichalcum, Hadron, the four Ruin abilities, sand/snow defence. **WRONG STAGE** for Thick Fat / Heatproof / Purifying Salt / Water Bubble (see §2)
4	**base damage** `tr(tr(tr(tr(2L/5+2)*bp*A)/D)/50)`	`:1718`	integer	`:2246` `Math.floor(Math.floor(22*mvBP*A/D)/50)+2`	**SAME** — at L50 `2L/5+2 = 22` , and `22*bp*A` is already an integer so the extra `tr` is a no-op
5	**+2**	`:1731`	addition	folded into `:2246`	SAME
6	**spread x0.75**	`:1737`	`modify`	`:2247` `md4096(base, 0.75)`	**SAME STAGE, SAME ARITHMETIC.** Measured: spread arm 13/282 disagree, control 13/282 — the same rows. Spread adds zero error
6b	**Parental Bond** (2nd hit x0.25)	`:1742`	`modify` , per hit	`:2337-2341` one hit x1.25 at the base stage	STRUCTURALLY DIFFERENT — declared in the engine's own comment; a two-hit move rolled once is not this stage's problem
7	**weather** `WeatherModifyDamage`	`:1746`	priorityEvent -> chainModify	`:2250-2251` `md4096`	**SAME STAGE.** Charizard Flamethrower -> Snorlax: 61 clear, 91 sun, 30 rain, both engines
8	**CRIT — a plain `tr(x * 1.5)` , NOT a modifier**	`:1748-1752`	`tr()`	`:2276-2281` (certain crits) / `:5912-5918` **(rolled crits, at the hit site)**	**SPLIT — see §3.** dmgRange's certain crit is at the right stage and passes. The battle loop's rolled crit is applied AFTER everything
9	**the randomizer — also NOT a modifier**	`:1755` , `battle.ts:2388` `tr(tr(d*(100-random(16)))/100)`	`tr()`	`:2528` `Math.floor(base*r/100)` inside `roll()`	**SAME POSITION, SAME ARITHMETIC.** Different SHAPE (11 uniform integers vs 16 inverted indices) — already documented in `engine/game_differential.js` ; the POSITION is not also different. Both endpoints match exactly
10	**STAB** (+ `ModifySTAB` for Adaptability)	`:1789-1792`	`modify`	`:2391-2392` , applied at `:2529` `md4096(d, stab)`	**SAME STAGE.** Adaptability x2 via `stabBoost` agrees (132 vs 132)
11	**type effectiveness**, clamped -6..6, `x2` per step up / `tr(/2)` per step down	`:1796-1812`	literal	`:2530` `Math.floor(d*eff)`	**SAME.** `floor(d/4) === floor(floor(d/2)/2)` for every integer, so the single floor is the reference. No clamp in ours, but no move reaches +-7 steps
12	**burn x0.5**, physical, not Guts, not Facade	`:1816-1820`	`modify`	`:2405-2406` , applied at `:2531` `md4096(d, burn)`	**SAME STAGE.** Measured burn arm 8/160 (5.0%) against a control of 4.6% — inside the control's own residual
13	**ModifyDamage chain** — the final item/ability chain	`:1826`	chainModify, spent once	`:2418-2419` `mod` / `MODMUL` , spent at `:2533` `mdChain`	**SAME STAGE AND GENUINELY A CHAIN.** Life Orb, Expert Belt, resist berries, Multiscale/Filter/Solid Rock/Prism Armor/Ice Scales/Punk Rock-defensive, Tinted Lens, Neuroforce,

#	stage	authority (line)	how	ours (frozen line)	verdict
					screens. Measured at the control's residual — see §4
13b	**Friend Guard** (`onAnyModifyDamage`)	`:1826` , same chain	chainModify, **in the same chain**	`:5892-5898` , `md4096` on the already-spent number	**RIGHT STAGE, WRONG CHAIN** — see §3
14	**bypassProtect x0.25**	`:1830`	`modify` , after the chain is spent	`:5931` `md4096(dmg, 0.25)`	SAME — a separate spend is correct here
15	**minimum 1**	`:1838`	`return 1`	absent	ABSENT, **and never observed**: 600 random (attacker, move, defender) draws produced 0 rows where Showdown floored to 1 and we returned 0. Recorded, not ranked
16	**16-bit truncation**	`:1841` `tr(baseDamage, 16)`	`tr()`	absent	ABSENT and unreachable — needs a damage above 2^16

2. EVERY MULTIPLIER WE APPLY, CLASSIFIED BY THE AUTHORITY'S STAGE

This is the table that outlives the audit. **Stage read from the handler's own event name via `Dex.forFormat('gen9championsvgc2026regmb')` , never from memory.**

The `uses` column is the tag artifact's sheet or click count. It is a **prior, not truth**, and it is read from the LIVE artifact while the engine bytes are read from the frozen release — deliberately, because usage is a fact about the corpus and not about the engine, and the corpus has grown since the release was cut. (The frozen copy's figures are lower across the board; nothing in the ranking moves.)

2a. WRONG STAGE — we apply it later than the authority does

Ordered by exposure — class size times usage, not usage alone, which is why eighteen items at a few thousand sheets each outrank one ability at 678. The measured column is the disagreement rate against Showdown over random (attacker, move, defender) triples with flat bodies, top roll, rows dropped when the reference KO'd (clamped) or dealt 0. **The control — the same rows with nothing switched on — disagrees on 4.1% (12/294).** That is the floor: anything at 4% is adding nothing, anything at 35% is the stage.

PER-ITEM USAGE FOR THE TYPE-ITEM ROW IS READ LIVE FROM `data/tags.json` AND IS DELIBERATELY NOT RESTATED IN THE ROW. The counts that used to stand there were correct when written and had all moved by the next gate run, because the tagger regenerates that artifact against a corpus that grows hourly — roughly a quarter of all tagged entities' `uses` moved in the single regeneration of 2026-08-10 (the exact tally is in `docs/MEDICHAM-SPRINT-NOTES.md` , where it is stated once). Membership is what matters and it is derived, not typed.

(The citation moved out of the table row on 2026-08-10, and the reason is worth one sentence. A table row is its own paragraph to `tests/test-docs-current.js` , so naming `data/tags.json` inside the row was a promise that every figure in that row came from it — and the row's other figures are DISAGREEMENT RATES from the run described above, which no artifact holds. `65.0%` passed the citation check for months purely because some unrelated `0.65` sat in `tags.json` , and it stopped passing the moment that number moved. The measurement is unchanged; only the false citation is gone.)

multiplier	authority event	ours (frozen line)	uses	measured disagreement
the 18 type items — every member of `damageMultType` , headed by Fairy Feather, Black Glasses, Mystic Water and Charcoal, with a long tail down to Silver Powder	`onBasePower` x1.2	`:2499-2500` `damageMultType` in the ModifyDamage chain	**the biggest class here**	Black Glasses **65.0% (13/20)**, Charcoal **40.0% (10/25)**
Tough Claws	`onBasePower` `[5325,4096]`	`:2313-2320` `boostsMoveClass` , at the base stage	627	**34.0% (54/159)**
Technician	`onBasePower` x1.5, priority 30	`:2308` , at the base stage	678	**40.3% (31/77)**
Sharpness	`onBasePower` x1.5	`:2313-2320`	314	**48.0% (12/25)**
Sheer Force (the x1.3 half)	`onBasePower` `[5325,4096]`	`:2326-2332` `removesOwnSecondaries.powerMult`	176	staged rows agree; same class, same fix
Thick Fat / Heatproof / Purifying Salt	`onSourceModifyAtk` / `onSourceModifySpA` x0.5 — the **STAT** stage	`:2487-2493` `halvesTypeDamage.attackerStatMult` in the ModifyDamage chain	136 / 17 / 60	**73.1% (19/26)**
Water Bubble (attacking x2 on Water)	`onModifyAtk` / `onModifySpA` — the **STAT** stage	`:2494` in the ModifyDamage chain	131	**77.3% (17/22)**
Iron Fist	`onBasePower` `[4915,4096]`	`:2313-2320`	114	9.5% (2/21)
Dry Skin (x1.25 taken from Fire)	`onSourceBasePower` x1.25	`:2487-2493` `halvesTypeDamage.basePowerMult`	133	**40.0% (10/25)**
Supreme Overlord	`onBasePower` , table `[4096,4506,4915,5325,5734,6144]`	`:2249` `boostsFromFallen` , `md4096(base, 1+0.1n)`	84	staged rows agree at n=1,3; the table is exact 4096ths and `1+0.1n` is not
Helping Hand	`onBasePower` `chainModify(1.5)`	`:5902` `Math.floor(d * 1.5)` on the rolled range, at the hit site	**4306**	**5/5 rows wrong** — Alakazam Psychic -> Snorlax: Showdown 108, ours 109; Kingambit Kowtow -> Snorlax 154 vs 157; Pikachu Thunderbolt -> Snorlax 49 vs 51
Expanding Force / Rising Voltage	`onBasePower` `[5325,4096]`	`:2303-2307` `terrainScaled` , `Math.floor(base*mult)` at the base stage	204 / 123	same class; also a plain float multiply rather than 4096ths

multiplier	authority event	ours (frozen line)	uses	measured disagreement
Muscle Band / Wise Glasses	`onBasePower` `[4505,4096]`	`:2516-2517`, hardcoded names in the ModifyDamage chain	112 / 33	Muscle Band **39.6% (74/187)**, Wise Glasses **42.2% (43/102)**
Mega Launcher / Strong Jaw / Punk Rock-offensive	`onBasePower`	`:2313-2320`	29 / 6 / 0	Strong Jaw 25.0% (2/8), Punk Rock 20.0% (1/5)
the -ate abilities' x1.2 (Pixilate 2875, Refrigerate 9, Aerilate/Galvanize/Dragonize/Normalize 0)	`onBasePower` `[4915,4096]`, priority 23	`:1985` `Math.floor(mvBP * damageMult)` — **right stage, wrong rounding** (floor vs round-half-up on 4096ths)	2875	not measurable through `moveHit` (see §5); the retype half is at the right stage and works
Sniper	`onModifyDamage` x1.5	`:2279-2280` and `:5916-5917`, folded into the crit's plain multiply	50	**34.8% top roll, 54.1% bottom roll**

The arithmetic, in full, for the row that started this

Kingambit Kowtow Cleave (Dark, physical, 85 BP) into Charizard. Bodies flat in both engines: atk 155, def 98, Charizard maxhp 153. Top roll, no crit, no burn, nothing else on the field.

```
SHOWDOWN   bp 85
           Black Glasses  modify(85, x1.2)   -- the BasePower chain           = 102
           base           tr(tr(tr(22 * 102 * 155)/98)/50) + 2                = 72
           randomizer     tr(tr(72*100)/100)                                  = 72
           STAB           modify(72, x1.5)                                    = 108
           type Dark vs Fire/Flying = 1x, ModifyDamage chain empty            -> 108

OURS       bp 85          (Black Glasses is not read here at all)
           base           floor(floor(22 * 85 * 155 / 98) / 50) + 2            = 61
           roll           floor(61 * 100 / 100)                                = 61
           STAB           md4096(61, 1.5)                                      = 91
           type 1x, burn 1x
           ModifyDamage   the x1.2 lands HERE: mdChain(91, ch4096(4096, 1.2))   -> 109
```

Same multiplier, same fixed-point helpers, **one stage apart, and a point of damage out**. The x1.2 applied to 85 gives 102, which is a base power; applied to 91 it gives 109, which is a damage. In between sit `tr(.../98)` and `tr(.../50)`, and neither commutes with a multiply.

Two failure modes hide inside "wrong stage", and both need fixing together.

1. **The stage itself.** A base power passes through `tr(.../D)` and `tr(.../50)` before it becomes damage; a final multiplier does not. The truncations do not commute.
2. **The chain.** Showdown folds every `onBasePower` handler into ONE `event.modifier` and spends it once. Ours applies each as its own `Math.floor`. Measured directly: Gallade Drain Punch -> Snorlax with **Iron Fist + Muscle Band** — Showdown 228, ours 227, while each alone agrees. A fix that moves these to the base-power stage but keeps one floor per member will still be wrong when two co-occur.

2b. ABSENT — we apply nothing at all

multiplier	authority event	evidence	usage
field terrain damage — Electric Terrain x1.3 on Electric, Psychic Terrain x1.3 on Psychic, Grassy Terrain x0.5 on Earthquake/Bulldoze/Magnitude, Misty Terrain x0.5 on Dragon	`onBasePower` `[5325,4096]` / `chainModify(0.5)` on the terrain CONDITION	grep of the frozen file: the only terrain reads in `dmgRange` are Hadron Engine (`:2230`) and the per-move `terrainScaled` tag (`:2305`). Measured: Pikachu Thunderbolt -> Snorlax **43 vs 34**; Hatterene Psychic -> Snorlax **94 vs 73**; Garchomp Earthquake in Grassy **60 vs 118**; Dragon Claw in Misty **92 vs 165**	Psychic Terrain 128 clicks + Psychic Surge 2; Electric 11; Grassy 11; Misty 9. Small today, and it is the whole of what a terrain team does
Fairy Aura / Dark Aura / Aura Break	`onAnyBasePower` `[5448,4096]` / `[3072,4096]`	`grep auraBoost` = 0 hits in the frozen engine, and no `aurabreak` entry in the tag artifact	Gardevoirite 412 sheets x every Fairy click on the field. **§0**
Charge (x2 on the user's next Electric move)	`onBasePower` `chainModify(2)` on the volatile	Pikachu Thunderbolt -> Snorlax with Charge: **66 vs 34**	move 1 click, but Electromorphosis applies it too
the whole `damageBoost` tag — Steelworker, Transistor, Dragon's Maw, Rocky Payload, Stakeout, Analytic, Reckless, Rivalry, Flare Boost, Toxic Boost, Sand Force, Gorilla Tactics, Hustle	`onBasePower` (Analytic, Reckless, Rivalry, Sand Force) or `onModifyAtk/SpA` (Stakeout, Steelworker, Transistor, Dragon's Maw, Rocky Payload, Hustle, Gorilla Tactics)	`grep damageBoost` in the frozen engine returns **one hit, and it is inside a comment**. 44 abilities carry the tag; nothing reads it	Reckless 77, Rivalry 39, Analytic 14, rest 0 on this corpus
Battery / Power Spot / Steely Spirit (ally base-power boosts)	`onAllyBasePower`	absent from the engine; Battery and Power Spot have no tag entry at all, Steely Spirit is `untagged`	0 on this corpus; they are doubles abilities and the corpus is doubles
Punching Glove	`onBasePower`	absent	`isNonstandard: 'Past'` — **banned in this format.** Recorded so nobody wires it
the 17 plates, the orbs, Soul Dew	`onBasePower`	absent	all `isNonstandard: 'Past'` — banned. The type-item cousins in 2a are the legal ones
Collision Course / Electro Drift / Brine / Retaliate	move `onBasePower`	not in `MC.moves` at all — a move-table gap, not a stage gap	filed for whoever owns `build_engine_data.js`

2c. SAME STAGE — checked, and correct

This list exists so the next session does not re-audit it. Each was measured, not read.

multiplier	authority event	ours	evidence
Life Orb x1.3	`onModifyDamage` `[5324,4096]`	`:2411-2412` , folded into `mod` via `ch4096`	**3.9% (11/284)** against a control of 4.1% — the same rows
Multiscale / Shadow Shield x0.5 from full	`onSourceModifyDamage`	`:2437-2451` `damageReduce`	**4.1% (12/294)** — identical to the control
Tinted Lens x2 on resisted	`onModifyDamage`	`:2453`	**4.1% (12/294)** — identical to the control
Filter / Solid Rock / Prism Armor x0.75 on SE	`onSourceModifyDamage`	`:2437-2451`	single rows agree exactly
Ice Scales x0.5 special	`onSourceModifyDamage` (**not** a stat modifier, which is the natural mis-statement)	`:2437-2451`	Alakazam Psychic -> Snorlax 36 vs 36
Punk Rock defensive x0.5 sound	`onSourceModifyDamage`	`:2437-2451`	Hyper Voice -> Snorlax 24 vs 24 (control 49)
Expert Belt x1.2 on SE	`onModifyDamage` `[4915,4096]`	`:2502-2503`	agrees
the resist berries x0.5	`onSourceModifyDamage`	`:2514-2515`	Chople on a SE Fighting hit, 114 vs 114
Neuroforce x1.25 on SE	`onModifyDamage`	`:2452`	agrees. (`neuroforce` is absent from `data/tags.json` and is name-wired — correct today, brittle)
Reflect / Light Screen / Aurora Veil `[2732,4096]` in doubles	`onAnyModifyDamage`	`:2462-2466` `DOUBLES_SCREEN = 2732/4096`	constant matches the authority's doubles branch exactly
the four Ruin abilities x0.75	`onAnyModifyDef` / `onAnyModifyAtk` / `onAnyModifySpA` / `onAnyModifySpD` — **STAT** stage	`:2242-2245` `md4096` on A or D	Sword of Ruin 81 vs 81, Tablets of Ruin 46 vs 46
sand Rock SpD x1.5 / snow Ice Def x1.5	`onModifySpD` / `onModifyDef` using `this.modify`	`:2208-2209` `md4096(D, 1.5)`	correct event, correct arithmetic
Huge Power / Pure Power x2, **Guts** x1.5, **Solar Power**, **Orichalcum Pulse**, **Hadron Engine** `[5461,4096]`	`onModifyAtk` / `onModifySpA`	`:2225-2230`	Huge Power row agrees (clamped, but the pre-clamp ratio is exact)
Choice Band / Choice Specs / Assault Vest x1.5	`onModifyAtk` / `onModifySpA` / `onModifySpD`	`:2192-2194`	right stage — **and all three are** `isNonstandard: 'Past'` **, so they are dead code in this format**
Adaptability x2 STAB	`onModifySTAB`	`:2391-2392` `stabBoost`	132 vs 132
spread x0.75	`modify` at `:1737`	`:2247`	13/282, the control's own rows
burn x0.5	`modify` at `:1818`	`:2531`	5.0% against a 4.6% control
Facade's burn exemption	`move.id !== 'facade'` at `:1817`	`:2405` keyed on `conditionalPower.when === 'userStatused'`	shape-keyed, membership printed, exactly one move
Technician's <=60 gate	`this.modify(bp, this.event.modifier)` at `:1650`	`:2308` gates on the raw `mvBP`	**EQUIVALENT, and this was nearly filed as a bug.** Technician's `onBasePowerPriority` is **30, the highest in the format**, so `event.modifier` is still 1 when its gate runs and `modify(bp, 1) === bp` . Proved: Body Slam (85 BP) + Technician + Silk Scarf gets no Technician boost in either engine
the randomizer's POSITION	`:1755`	`:2528`	both endpoints match on every control row
the certain crit's position	`:1751`	`:2276-2281`	Frost Breath and Storm Throw agree at BOTH endpoints
the base-damage formula itself	`:1718`	`:2246`	a 294-row control at 4.1% residual, and the 12 failures are named moves (Beak Blast, Night Daze, Spirit Shackle, Trop Kick, Fickle Beam, Apple Acid), not arithmetic

3. THE TWO NON-MODIFIERS, CHECKED EXPLICITLY

The crit: our arithmetic is right, our POSITION is wrong in the battle loop

- **Is it a plain truncated x1.5, or did it go through the 4096ths helper?** Plain. `:2280` `Math.floor(base*1.5*critMult)` and `:5917` `Math.floor(dmg*1.5*critMult)` . Neither touches `md4096` . That matches the authority's `tr(baseDamage * 1.5)` and its "crit - not a modifier" comment. **Correct.**
- **Position.** The authority puts the crit at `:1751` — *before* the randomizer, STAB, the type chart, burn and the ModifyDamage chain. `dmgRange` puts its certain crit in exactly that place (`:2276` , after spread and weather, before `roll()`) and it passes at both endpoints. The **battle loop's rolled crit** (`:5912-5918`) multiplies the number that has already been rolled, STAB'd, type-charted, burnt and chain-spent.

At the TOP roll the randomizer is the identity, so the error is smaller and the wrongness looks like nothing. Measured over random triples:

arm	disagree
no crit, bottom roll (**the control**)	20/364 — **5.5%**
crit, top roll	72/346 — 20.8%
crit, bottom roll	165/355 — **46.5%**
crit + Life Orb, bottom roll	210/340 — **61.8%**
crit + Sniper, top roll	109/313 — 34.8%
crit + Sniper, bottom roll	179/331 — 54.1%

Sniper compounds it twice: it is `onModifyDamage` , so it belongs in the final chain, and ours folds it into the crit's plain multiply.

The roll: same position, different shape

- **Position: SAME.** `:2528 Math.floor(base * r / 100)` sits between the crit and STAB, which is where `battle.ts:2388 randomizer` sits. Confirmed at both endpoints on every control row.
- **Shape: different, and already documented** in `engine/game_differential.js`'s header — 11 uniform integers here against 16 inverted indices there, agreeing only at the endpoints. **That note does not cover position, and position is fine.** The shape question is not this audit's.

---

4. THE CHAIN, NOT ONLY THE STAGE — FRIEND GUARD

Friend Guard is `onAnyModifyDamage`, so it belongs in the **same chain** as Life Orb, the screens, the resist berries and Expert Belt. Ours applies it at `:5896` as its own `md4096` on the number `dmgRange` has already spent its chain on.

Right stage, wrong chain, and it costs a point on a fifth of values. Two spends against one, over base damages 20..300:

```
Life Orb x1.3 then Friend Guard x0.75
  authority  modify(d, chain(chain(1, 1.3), 0.75))    one spend
  ours       modify(modify(d, 1.3), 0.75)             two spends
  -> 60 of 281 base-damage values disagree (21.4%);  d=45: authority 44, ours 43
```

Friend Guard is **1015 sheets**. Helping Hand (§2a) has the same double-spend problem on top of its stage problem.

---

5. WHAT THIS AUDIT COULD NOT SEE, SAID OUT LOUD

The harness calls `battle.actions.moveHit`, which is one level below `spreadMoveHit`. Three events never fire there, so three things are argued from the handler source rather than measured:

- `ModifyMove` — so Sheer Force's `hasSheerForce` and the -ate abilities' `typeChangerBoosted` are never set by Showdown. Sheer Force was staged by hand; **the -ate abilities could not be** and their row above is reasoned from `data/abilities.ts`, not measured.
- `spreadHit` and `willCrit` — both staged by hand, and the staging is stated in the probe.
- **the hit loop**, so `move.hit` and multi-hit are out of scope. `tests/test-engine-diff.js` already records this boundary.

Two further limits, stated because they are the shape of the control failures this project keeps finding:

- **The reference clamps at the defender's HP and we do not.** Four rows in the first pass read as disagreements purely because Showdown had KO'd. Every rate above drops those rows.
- **Both abilities are set explicitly on both sides, always.** The first pass left the defender at its species default and measured Araquanid's own Water Bubble against a blank, and Heliolisk's own Dry Skin against a blank. That is the Choice-Scarf-against-a-Choice-Scarf failure, and it happened here before it was caught.

---

6. THE ORDER TO FIX IN

1. **Fairy Aura / Dark Aura at the BasePower stage, with `[5448,4096]` and `[3072,4096]`.** Another agent is wiring `auraBoost` now. §0 is the acceptance test: 8/8, not 2/8.
2. **The 18 type items** (the largest class in 2a) — one line, `damageMultType` moves from `MODMUL` to a base-power chain. Everything below shares that chain, so build it once.
3. **Helping Hand** (4306 clicks) — and it needs the chain as well as the stage.
4. **Technician, Tough Claws, Sharpness, Sheer Force, Iron Fist, Mega Launcher, Strong Jaw, Punk Rock, Supreme Overlord, Expanding Force / Rising Voltage, Muscle Band, Wise Glasses, Dry Skin** — the same move into the same chain.
5. **Thick Fat / Heatproof / Purifying Salt / Water Bubble** into the STAT stage, beside the Ruin abilities that already live there.
6. **The battle loop's rolled crit** — it must be applied inside `dmgRange`, before the roll, not to `dmgRange`'s output. This is the largest single measured effect in the document (46.5% -> 61.8%).
7. **Friend Guard** into the ModifyDamage chain rather than beside it.
8. **The field terrain multipliers**, which are absent entirely.
9. ~~`damageBoost` — 44 abilities carry it and nothing reads it.~~ **DONE, in two passes.** ROADMAP #92 (3.7x) wired the unconditional, type-naming half — five abilities, all 0 corpus uses. ROADMAP #112 (3.83.0) made the HP condition machine-readable and added the other four: Blaze, Torrent, Overgrow, Swarm — 9,141 uses. Solar Power is still refused, correctly: its condition is a WEATHER and `inWeather` is a separate clause. The membership was printed before either wiring, and `tests/test-damage-stages.js` re-derives it every run.

Every one of these needs a failing probe in `tests/test-mechanics.js` first. None of them is open work until it has one.